

ENGR-E 312 Lab 8: Romi Autonomous Vehicle with Line Following, ArUco Road Sign Detection, and Drive-by-Video Interface

Jaiden Medina & Brady Adams

TABLE OF CONTENTS

Title Page	1
Table of Contents	2
Abstract	4
I. Introduction	4
A. Project Objective	4
B. System Requirements	4
C. Project Scope	5
II. System Description	5
A. Overall System Overview	5
B. Manual and Autonomous Operation	5
C. Line Following Functionality	5
D. ArUco Road Sign Detection	5
E. Drive-by-Video Interface	5
F. Operator Feedback Interface	6
III. Software Design	6
A. Software Architecture	6
B. Software Architecture Diagram	6
C. Arduino-Level Control Software	7
D. Raspberry Pi Vision Software	7
E. Curses-Based Operator Interface	7
F. Road Sign Decision Logic	7
IV. Hardware and Mechanical Design	7
A. Romi 32U4 Platform	7
B. Reflectance Sensor Array	7
C. Raspberry Pi and Camera	8
D. Camera Mount	8
V. Testing and Results	8
A. Line Following Performance	8
B. ArUco Detection Performance	8
C. Drive-by-Video Performance	8
D. Operator Interface Feedback	8
E. Video Resolution Testing	8
VI. Discussion and Limitations	8
VII. Future Improvements	9
VIII. Conclusion	9

ENGR-E 312 Lab 8: Romi Autonomous Vehicle with Line Following, ArUco Road Sign Detection, and Drive-by-Video Interface

Jaiden Francisco Medina

*Computer Engineering, Luddy School of Informatics,
Computing, and Engineering
Indiana University Bloomington
Bloomington, IN, USA
jfrmedina@iu.edu*

Brady Michael Adams

*Computer Engineering, Luddy School of Informatics,
Computing, and Engineering
Indiana University Bloomington
Bloomington, IN, USA
brmadam@iu.edu*

Abstract—This project presents the design and implementation of a Romi-based autonomous vehicle system capable of manual operation, autonomous line following, visual road sign detection, speed control, and drive-by-video feedback. The system integrates a Pololu Romi 32U4 mobile platform, Raspberry Pi, Raspberry Pi Camera V2.1, reflectance sensor array, custom camera mount, Redis-based video streaming, OpenCV computer vision, and a curses-based operator interface. The robot can be manually driven to a line track, switched into autonomous mode, and controlled through a feedback loop that uses line sensor data to maintain stable path tracking. ArUco markers are used as visual road signs, where marker IDs command the robot to stop, switch to high speed, or switch to low speed. The laptop-side interface provides live video feedback, marker overlay visualization, and real-time robot status information. During demonstration, the system achieved smooth line following, clear high/low speed distinction, stop/resume behavior, and effective user interface feedback, earning a subtotal score of 90/100. Remaining limitations included intermittent video streaming and occasional missed ArUco marker detections. Future improvements include improved video stream stability, multi-frame marker confirmation, camera angle adjustment, and marker-distance estimation for more precise response timing. Overall, the project demonstrates a complete cyber-physical system that combines embedded control, computer vision, networking, mechanical design, and human-machine interaction into one integrated autonomous robotics platform.

Keywords—*autonomous vehicle, cyber-physical systems, line following, ArUco markers, computer vision, Raspberry Pi, Romi 32U4, Redis, OpenCV, embedded systems.*

I. INTRODUCTION

A. Project Objective

The objective of this final project was to design and implement a complete autonomous Romi vehicle

system that could follow a line track and respond to visual ArUco “road signs” designed by the teaching assistant. The project combined multiple earlier lab assignments into the final integrated cyber-physical system, similar to the class name “Advanced Cyber-Physical Systems”. These included manual robot control automatic line following, a custom camera mount, i.e. “frictionless design”, drive-by-video streaming, and computer vision-based road sign detection.

The robot was required to begin under manual control, allowing the operator to drive it to the black electrical tape line track. Once the robot detected the line using the infrared sensor, the operator could calibrate the sensors automatically and then switch into autonomous mode. In autonomous mode, the robot continuously followed the line while also using a camera to detect ArUco markers placed around the course. Each marker represented a specific road sign command: stop, high speed, or low speed.

B. System Requirements

The final system was designed to meet the major project requirements. The robot needs to include manual and automatic control through a curses interface, a camera mount, a drive-by-video interface, line-following behavior, high-speed and low-speed operation, stop/resume behavior, and automatic ArUco responses. The project prompt specified that marker ID 5 should cause the robot to stop, marker ID 6 should cause high-speed operation, and marker ID 7 should cause low-speed operation.

The system also needs to provide operator feedback.

The video interface was expected to show the detected ArUco marker with a visible overlay, while

the curses interface was expected to show the current mode, marker detection, and robot action.

C. Project Scope

The scope of the project included hardware design, software design, control system design, computer vision, video streaming, and mechanical design. The completed system used the Romi 32U4 robot as the mobile platform, the Raspberry Pi as the higher-level processing unit, the Raspberry Pi Camera V2.1 for vision, and a laptop as the operator station.

The project was not only focused on getting the robot to move. It was focused on integrating multiple subsystems into one working cyber-physical system. This included sensing, actuation, feedback control, computer vision, communication, mechanical support, and human-machine interaction.

II. SYSTEM DESCRIPTION

A. Overall System Overview

The final project system is a complete cyber-physical robotics platform built around the Pololu Romi 32U4 robot, a Raspberry Pi, a Raspberry Pi Camera V2.1, a custom camera mount, i.e. “Frictionless Design”, and a laptop-side operator interface. The system combines physical robot motion with software-based control, computer vision, and real-time operator feedback.

The robot has two main operation modes: manual mode and autonomous mode. When the robot is in manual mode, the operator controls the robot using the laptop keyboard through a curses interface. When the robot is in autonomous mode, the robot follows a line using its reflectance sensor array and responds to visual road signs detected by the camera.

The system was built from earlier labs in the course. Lab 5 provided a curses-based manual and automatic driving interface. Lab 6 added the customized camera mount. Lab 7 added the drive-by video system using the Raspberry Pi camera, Redis, and OpenCV. The final integration added ArUco marker detection and road-sign response behavior.

B. Manual and Autonomous Operation

Manual mode allows the operator to drive the robot directly. This is useful for the beginning of the demo because it allows the robot to be driven to its starting location at the line track, as specified in the project description. The curses interface provides keyboard control, allowing the operator to move the robot forward, backward, left, right, and then use the spacebar to stop.

Once the robot is positioned correctly on the line, the operator can press the “c” button to prompt the robot to automatically calibrate itself, the switch to

autonomous mode. While the robot is in autonomous mode, the robot uses line sensor feedback to remain centered on the track. If the robot loses the line, the intended behavior is for the robot to reposition itself on the line, centered on the infrared sensors using a PID loop. The operator can also return to manual mode so the operator to reposition it safely if required.

C. Line Following Functionality

The robot uses a reflectance sensor array to detect the line. The sensor readings are converted into a line position estimate. That position estimate is then converted into a line error, which tells the robot whether it is drifting left or right.

The line-following behavior was one of the strongest parts of the final system. During the demo, the robot received full credit for line following, earning 15/15. The demo feedback described the line following as smooth and stable. This shows that the sensor calibration, error calculation, and motor correction logic worked well together.

D. ArUco Road Sign Detection

The system uses ArUco markers as visual road signs. The Raspberry Pi Camera V2.1 captures frames, and the computer vision software searches for markers in those frames. When a marker is detected, the system reads its ID and maps it to a robot action.

The three road sign behaviors can be seen in **Table 1** below.

Marker ID	Meaning	Robot Action
5	Stop Sign	Stop, pause, then resume
6	High-Speed Sign	Switch to high-speed operation
7	Low-Speed Sign	Switch to low-speed operation

Table 1: ArUco Sign Markers Meanings

During the final demo, ArUco detection earned 8/10. The detection worked, but the demo notes state that the system missed yellow and red markers a few times. This means the vision system was functional but not perfectly consistent.

E. Drive-by-Video Interface

The drive-by-video interface allows the operator to view live video feed from the robot’s camera on the laptop. The Raspberry Pi captures camera frames and sends them through Redis. The laptop then reads the frames and displays them using OpenCV.

The drive-by-video interface is important because it allows the operator to drive or monitor the robot from the robot’s point of view. It also supports marker visualization by showing the ArUco marker detected with a visible overlay and marker ID.

During the final demo, this subsystem was the weakest area. The drive-by-video interface earned 5/10 because the video was intermittent. The system worked, but the stream was not stable enough to receive full credit.

F. Operator Feedback Interface

The curses interface provides real-time feedback to the operator. It shows the current robot mode, motor commands, line error, battery voltage, calibration status, marker detection, and robot action. This feedback makes the system easier to control and debug.

During the final demo, the UI feedback earned 5/5.

The feedback clearly showed the mode, marker detection, and actions. This was important because the operator could tell what the robot was doing without needing to guess from motion alone.

III. SOFTWARE DESIGN

A. Software Architecture

The software architecture for the Lab 8 Romi autonomous vehicle was divided across three main computing layers: the Romi 32U4 microcontroller, the Raspberry Pi, and the laptop operator station.

Each layer had a specific responsibility in the complete cyber-physical system. The Romi 32U4 handled low-level embedded control, including motor commands, encoder feedback, reflectance sensor readings, sensor calibration, and PID-based line following. The Raspberry Pi handled higher-level vision processing through the Raspberry Pi Camera V2.1, OpenCV, ArUco marker detection, and Redis communication. The laptop acted as the operator station, displaying the curses-based control interface and the drive-by-video stream.

This division was important because the robot had to perform real-time motion control while also processing camera frames and communicating with the operator. The line-following loop needed to stay responsive so the robot could remain centered on the black tape track. At the same time, the camera system needed to continuously search for ArUco road signs and send marker information to the control software. By separating the software into multiple modules, the system could handle movement, vision, communication, and user feedback without placing all responsibilities on one program.

The main software files used in the final Lab 8 implementation were Lab8_E321.ino, Project_driver.py, server1.py, and client.py. The Arduino program controlled the Romi hardware directly. The driver program managed the operator

interface, manual/autonomous mode switching, PID values, speed commands, and ArUco road sign actions. The server program ran on the Raspberry Pi and captured camera frames, detected ArUco markers, and published the latest frame and marker state through Redis. The client program read the Redis frame data and displayed the live camera view on the laptop. The repository confirms that the Lab 8 code is split into ArUco server/client programs and movement/UI programs, matching this architecture.

B. Software Architecture Diagram

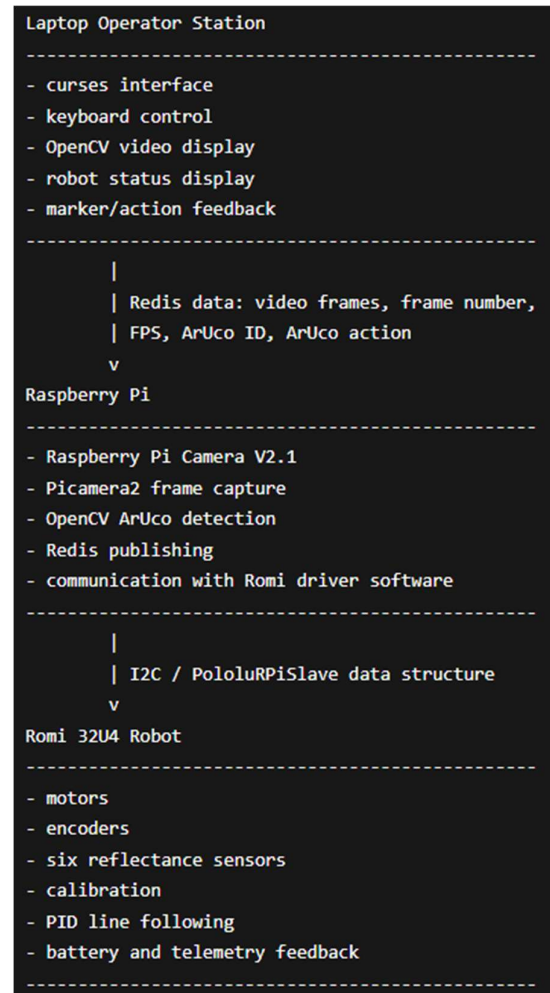


Figure 1: Lab 8 Software and Communication Architecture

As seen in **Figure 1**, the architecture diagram shows how the system connected the physical hardware to the software component, which was running on the Raspberry Pi and laptop. The Romi 32U4 handled the direct embedded control layer, while the Raspberry Pi acted as the vision and communication layer. The laptop was used as the operator interface layer. This architecture supported the main project requirements because the robot could be manually driven, switched

into autonomous mode, follow the line using sensor feedback, detect ArUco road signs using camera feedback, and display real-time robot status to the operator.

C. Curses-Based Operator Interface

The Arduino program controlled low-level robot behavior. This included reading the line sensors, calibrating the sensors, controlling the motors, and running the PID line-following system. The reflectance sensors were used to find the position of the black line under the robot. From this position, the program calculated a line error.

The line error told the robot if it was drifting left or right. The PID controller then used this error to adjust the left and right motor speeds. If the robot moved too far to one side, one motor would speed up and the other would slow down so the robot could move back toward the center of the line.

This was important because line following was one of the main requirements of the final project. During the demo, this part worked well and the robot was able to follow the line smoothly.

D. Raspberry Pi Vision Software

The Raspberry Pi handled the camera and ArUco marker detection. The Pi camera captured the video frames, and OpenCV was used to search for ArUco markers in each frame. When a marker was found, the program read the marker ID and matched it to a robot command.

The marker IDs used were:

Marker ID	Meaning	Robot Action
5	Stop	Stop, pause, then resume
6	High Speed	Switch to high-speed mode
7	Low Speed	Switch to low-speed mode

Table 3: ArUco Sign Markers Meanings

This allowed the robot to respond to visual road signs instead of only following the line. The detection worked during the demo, but it was not perfect because some markers were missed.

E. Curses-Based Operator Interface

The curses interface allowed the operator to control and monitor the robot from the home laptop. It had two different modes, manual and autonomous mode. In manual mode, the keyboard could be used to drive the robot forward, backward, left, and right. The operator could also stop the robot, calibrate the sensors, and switch into autonomous mode.

The interface also showed robot statistics. It showed the current mode, motor values, line error (which was important for debugging autonomous mode), battery voltage, calibration status, marker detection, and robot action. This was useful because the operator could see what the robot was doing in real time instead of only guessing from the robot's movement.

This also greatly improved debugging.

Overall, the interface worked well and helped make the system easier to control during the demo.

F. Road Sign Decision Logic

The road sign logic was used when the robot detected an ArUco marker. If the robot detected marker ID 5, it stopped for a short time and then started moving again. The stop signs also had a cooldown so the robot would not keep stopping on the same sign repeatedly.

If the robot detected marker ID 6, it switched to high-speed line following. If it detected marker ID 7, it switched to low-speed line following. This made the robot's behavior change based on what it saw through the camera.

This part connected the vision system to the robot control system. It showed that the robot was not only detecting markers but also using those detections to make movement decisions.

IV. HARDWARE AND MECHANICAL DESIGN

A. Romi 32U4 Platform

The main hardware platform was the Pololu 32U4 robot. The Romi provided motors, wheels, encoders, battery system, and control board. This platform was useful because it already supported both manual driving and autonomous movement.

Romi was responsible for the physical movement of the system. The motors allowed the robot to move around the track, while the sensors helped it stay on the line.

B. Reflectance Sensor Array

The robot used a reflectance sensor array to detect the track, which was made of black electrical tape. The infrared sensors measured the difference between the

dark line and the white floor. After calibration, the robot could use these values to estimate where the line was under the robot.

This was important because the line sensor data was used by the PID controller. Without accurate line readings, the robot would not be able to follow the track smoothly.

C. Raspberry Pi and Camera

The Raspberry Pi was used as the higher-level processing system. It worked with the Raspberry Pi Camera V2.1 to capture video and detect ArUco markers. The camera was also used for the drive-by-video system, which allowed the operator to see the robot's point of view.

The camera system was important because the robot needed to detect road signs during autonomous driving. The position and angle of the camera affected how well the robot could see the markers.

D. Camera Mount

I made a custom camera mount, termed "Frictionless Design", held the Raspberry Pi camera on the robot. This was very crucial because the camera needed to stay stable while the robot moved. If the camera moved too much or was pointed at an incorrect angle, the ArUco markers could be missed.

The mount connected the mechanical design part of the project to the software part. The software could only work correctly if the camera was physically mounted in a good position.

V. TESTING AND RESULTS

A. Line Following Performance

The line following performance was one of the strongest parts of the system. During the demo, the robot earned full credit for line following. The robot was able to stay on the track and correct itself while moving.

This shows that the sensor calibration, line error calculation, and PID controller were working together correctly. The robot did not just drive straight. It adjusted its motor speeds based on the line position.

B. ArUco Detection Performance

The ArUco detection system worked, but it was not perfect. The robot was able to detect markers and respond to them, but some markers were missed during the demo.

This could have happened because of lighting, camera angle, marker distance, or the robot moving past the sign too quickly. The system still showed that the robot could use visual input to control its behavior.

C. Drive-by-Video Performance

The drive-by-video system displayed the camera feed from the robot. This helped the operator see what the robot was seeing. The video feed worked, but it was intermittent during the demo.

The video performance was one of the weaker parts of the system. The video issue could have been caused by communication delay, Redis timing, frame size, or camera processing speed.

D. Operator Interface Feedback

The operator interface worked well during the demo. It showed the current mode, marker detection, robot action, battery information, and line error. This made it easier to understand what the robot was doing.

This part received full credit because the feedback was clear and useful. It helped with both controlling and debugging the robot.

E. Video Resolution Testing

Based on the results, the best resolution for this system was 320×240 . It had a good frame rate while still keeping enough image quality for marker detection. The 640×480 resolution was clearer but slower, and the 160×120 resolution was faster but had less detail, as seen in **Table 2**.

Resolution	FPS	Result
640×480	31.57 FPS	Best image quality, but slower
320×240	43.58 FPS	Best balance between speed and quality
160×120	46.39 FPS	Fastest, but lower image detail

VI. DISCUSSION AND LIMITATIONS

Overall, the final system worked well. The robot could be manually driven, automatically calibrated, and then switch into autonomous mode, follow the line, detect road signs, and change behavior based on the marker ID

The strongest parts of the system were the line following and the operator feedback. The robot followed the line smoothly, and the interface clearly showed what stats the robot was sensing.

The main limitations were the video stream and the marker detection. The video was intermittent, and the robot missed some ArUco markers. This shows that the vision system was working, but it still needed better stability. The marker detection could be affected by lighting, distance, camera angle, and how fast the robot was moving.

Another limitation was that many programs had to work together at the same time. If one part had a delay or issue, it could affect the full system. This made debugging more difficult.

VII. FUTURE IMPROVEMENTS

Future improvements include improving the video streaming could be improved by reducing the frame size or compressing the image before sending it from the client to the server.

The ArUco detection could also be improved by requiring the marker to be detected for multiple frames before the robot responds. This would help prevent false detections or repeated triggers.

The camera mount could also be adjusted to improve the viewing angle. If the camera could see the markers earlier, the robot would have more time to respond.

The PID controller could also be tuned separately for low-speed and high-speed mode. This would help the robot stay smoother when switching between different speeds.

VIII. CONCLUSION

The Lab 8 project successfully combined multiple parts of the course into one final autonomous robot system. The robot used manual control, sensor calibration, PID line following, camera-based ArUco detection, Redis communication, and a curses-based interface.

The final system was able to follow the line, detect road signs, stop, switch speeds, and show useful

feedback to the operator. The main problems were the intermittent video stream and occasional missed marker detections.

Overall, the project showed how hardware, software, computer vision, and control systems can work together in one cyber-physical system. It also showed that the hardest part was not just making one feature work, but getting all parts of the system to work together at the same time.